

1. Introduction

ArchDSL is a declarative Domain Specific Language (DSL) designed for software, data, and cloud architects. It addresses the common pain points of manual diagramming by providing a syntax that enforces structural rigidity, automatic layout distribution, and explicit dataflows, while maintaining human-readable source code.

The layout engine interprets nested blocks (`box` , `stack` , `layer`) and automatically calculates optimal bounds, distributing elements cleanly without the need for manual coordinate placement.

2. Core Primitives

2.1. Structural Containers

Containers group elements together and define the layout methodology for their children.

- `box "Name" as alias { ... }` : A standard container. Auto-sizes to fit its children based on its layout directive.
- `stack "Name" as alias { ... }` : A strict vertical or horizontal container designed specifically to hold `layer` elements. Used for reference architectures (e.g., OSI model, Data platforms).
- `layer "Name" as alias { ... }` : A semantic partition within a `stack` . Spans the full width (or height) of the parent stack.

2.2. Layout Directives

Layout directives instruct the rendering engine on how to position sibling nodes inside a container. They are declared at the top of a block using the `layout:` keyword.

```
box "Microservices" as ms {  
  layout: grid          // Options: horizontal, vertical, grid  
  direction: top-down   // Options: top-down, bottom-up, left-right, right-left  
  
  service "Auth" as auth  
  service "Cart" as cart  
}
```

2.3. Generic Nodes

General-purpose architectural nodes mapped to standard shapes:

- **component** : A generic system component (Box with component icon).
- **service** : A microservice or background process (Gear/Hexagon).
- **database** : Relational or general databases (Cylinder).
- **storage** : Object storage, buckets, or file systems (Database symbol with file badge).
- **queue** : Message brokers, event buses (Pipe/Stack).
- **node** : A generic compute instance, VM, or physical hardware (3D Box).
- **actor** : A user, client, or external persona (Stick figure).

3. Connections and Dataflows

Connections should generally be defined outside of the structural blocks to keep the layout definitions clean. The engine routes connections optimally without breaking the rigid container constraints.

3.1. Syntax Types

Different arrow types carry distinct semantic meanings, which map to line styles (solid, dashed, thickened) in the rendered output.

Syntax	Meaning	Visual Style
<code>-></code>	Standard connection / Control Flow / API Call	Solid line, standard arrow
<code>--></code>	Asynchronous or Weak connection	Dashed line, standard arrow
<code>=></code>	Dataflow (High volume / ETL)	Thick solid line, block arrow
<code>~></code>	Stream / Continuous connection	Wavy line, standard arrow
<code><-></code> / <code><=></code>	Bidirectional flow	Double-headed arrows

3.2. Connection Attributes

Attributes can be attached to connections using bracket notation `[...]`.

```
// Syntax: source [arrow] target : "Label" [attributes]
api_gw -> auth_service : "Validate Token" [color="red", style="dotted"]
spark_node => datalake : "Daily Batch Write" [weight="bold"]
```

4. Google Cloud Platform (GCP) Component Library

ArchDSL natively supports cloud provider icons via namespaces. The Google Cloud standard library uses the `gcp::[category]::[Service]` namespace. When declared, the rendering engine automatically applies the official Google Cloud icon, standard GCP colors, and appropriate shape styling.

Syntax Example:

```
gcp::database::CloudSQL "User DB" as sql_db
gcp::compute::GKE "Kubernetes Cluster" as k8s
gcp::analytics::BigQuery "Data Warehouse" as bq
```

4.1. Compute

Identifier	Service Name	Description
<code>gcp::compute::ComputeEngine</code>	Compute Engine	Virtual Machines, GPUs, TPUs
<code>gcp::compute::AppEngine</code>	App Engine	Fully managed serverless platform
<code>gcp::compute::CloudFunctions</code>	Cloud Functions	Event-driven serverless compute
<code>gcp::compute::CloudRun</code>	Cloud Run	Serverless containers
<code>gcp::compute::GKE</code>	Google Kubernetes Engine	Managed Kubernetes
<code>gcp::compute::BareMetalSolution</code>	Bare Metal Solution	Hardware for specialized workloads

4.2. Storage

Identifier	Service Name	Description
<code>gcp::storage::CloudStorage</code>	Cloud Storage	Object storage (Buckets)
<code>gcp::storage::PersistentDisk</code>	Persistent Disk	Block storage for VMs
<code>gcp::storage::Filestore</code>	Filestore	High-performance file storage
<code>gcp::storage::LocalSSD</code>	Local SSD	Locally attached block storage

4.3. Databases

Identifier	Service Name	Description
<code>gcp::database::CloudSQL</code>	Cloud SQL	Managed MySQL, PostgreSQL, SQL Server
<code>gcp::database::CloudSpanner</code>	Cloud Spanner	Globally distributed relational DB
<code>gcp::database::Bigtable</code>	Cloud Bigtable	NoSQL wide-column store
<code>gcp::database::Firestore</code>	Firestore	NoSQL document database
<code>gcp::database::Memorystore</code>	Memorystore	Managed Redis and Memcached
<code>gcp::database::AlloyDB</code>	AlloyDB	PostgreSQL-compatible database

4.4. Data Analytics & Streaming

Identifier	Service Name	Description
<code>gcp::analytics::BigQuery</code>	BigQuery	Serverless data warehouse
<code>gcp::analytics::Dataflow</code>	Dataflow	Stream and batch data processing
<code>gcp::analytics::Dataproc</code>	Dataproc	Managed Hadoop and Spark
<code>gcp::analytics::PubSub</code>	Pub/Sub	Messaging and event ingestion
<code>gcp::analytics::Looker</code>	Looker	Enterprise BI and analytics
<code>gcp::analytics::DataCatalog</code>	Data Catalog	Data discovery and metadata
<code>gcp::analytics::Composer</code>	Cloud Composer	Managed Apache Airflow workflow orchestration

4.5. Networking

Identifier	Service Name	Description
<code>gcp::network::VPC</code>	Virtual Private Cloud	Global virtual network
<code>gcp::network::CloudLoadBalancing</code>	Cloud Load Balancing	Global/Regional load balancing
<code>gcp::network::CloudCDN</code>	Cloud CDN	Content delivery network
<code>gcp::network::CloudDNS</code>	Cloud DNS	Domain name system
<code>gcp::network::CloudInterconnect</code>	Cloud Interconnect	Dedicated hybrid connectivity
<code>gcp::network::CloudRouter</code>	Cloud Router	Dynamic routing (BGP)
<code>gcp::network::CloudNAT</code>	Cloud NAT	Network address translation

4.6. AI & Machine Learning

Identifier	Service Name	Description
<code>gcp::ai::VertexAI</code>	Vertex AI	Unified ML platform
<code>gcp::ai::VisionAPI</code>	Cloud Vision API	Image analysis
<code>gcp::ai::SpeechToText</code>	Speech-to-Text	Audio transcription
<code>gcp::ai::TranslationAPI</code>	Translation API	Language translation
<code>gcp::ai::NaturalLanguage</code>	Natural Language API	Text parsing and analysis

4.7. Security, Identity & Management

Identifier	Service Name	Description
<code>gcp::security::IAM</code>	IAM	Identity and Access Management
<code>gcp::security::KMS</code>	Cloud KMS	Key Management Service
<code>gcp::security::CloudArmor</code>	Cloud Armor	DDoS and WAF protection
<code>gcp::security::IAP</code>	Identity-Aware Proxy	Zero-trust application access
<code>gcp::management::CloudMonitoring</code>	Cloud Monitoring	Infrastructure monitoring
<code>gcp::management::CloudLogging</code>	Cloud Logging	Log management

5. Comprehensive Example: GCP Serverless Data Pipeline

Below is a full example illustrating the structural rules, dataflows, and GCP components working together.

```

architecture "Real-Time User Analytics Pipeline" {

    actor "Mobile User" as user

    // VPC Boundary
    box "GCP Virtual Private Cloud (VPC)" as vpc {
        layout: vertical

        // Load Balancing Layer
        gcp::network::CloudLoadBalancing "Global HTTPs LB" as lb

        // Processing Layer
        stack "Compute & Ingestion" {
            direction: left-right

            layer "API Gateway" {
                gcp::compute::CloudRun "Ingestion API" as api
            }

            layer "Event Bus" {
                gcp::analytics::PubSub "Events Topic" as pubsub
            }
        }

        // Analytics Layer
        stack "Data Processing & Storage" {
            direction: left-right

            layer "Stream Processing" {
                gcp::analytics::Dataflow "Streaming ETL Job" as dataflow
            }

            layer "Persistence" {
                layout: vertical
                gcp::storage::CloudStorage "Raw Data Lake" as gcs
                gcp::analytics::BigQuery "Analytics DWH" as bq
            }
        }
    }

    // External BI Tool
    box "Business Intelligence" {
        gcp::analytics::Looker "Looker Dashboards" as looker
    }
}

```

```
// -----  
// Connections & Flows  
// -----  
  
// User flow  
user -> lb : "HTTPS JSON Payload"  
lb -> api : "Route traffic"  
  
// Core Dataflow (Thick arrows)  
api => pubsub : "Publish Event"  
pubsub => dataflow : "Subscribe / Stream"  
  
// Forked writes  
dataflow => gcs : "Backup Raw Events"  
dataflow => bq : "Write Aggregated Metrics"  
  
// Analytical reads  
looker -> bq : "Query Data" [color="blue", style="dashed"]  
}
```